



Pasted text(58).txt
Document



Pasted text (2)(7).txt
Document

leaned way to into the hermes thing i was more asking for it to just be a mention in the layout cause i didnt see persistant memory or any memory add in the first great plan u made heres both full conversations we had please try to take ur time and surgically and methodiocally distill this better and more presice and also add the structure around getting fable to plan it and lay it all out in a way that he can get codex gpt 5.5 and gemini pro 3.5 to do some of the building foundation as well to save blowing the whole budget making fable do the work

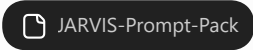


JARVIS-Prompt-Pack.pdf
PDF

Thought for 25s >

Yes, the PDF is useful, but it should be labelled correctly.

It is not the OS plan, it is the prototype/aesthetic inspiration layer.

The JARVIS pack is basically a 5-step skeleton: build a 3D markdown note plan, add a note-grounded chat brain, add a browser voice layer to source-proof, add personality, then add a reminder that it's a prototype endpoint. It also explicitly says the finished JARVIS layer will have hundreds of nodes and millions of references to help in the solution with things like a reactor, HUD, small calendar, location, and a few other report boards to keep things model-based, fun, and personal. 

For more info, see the PDF.

JARVIS Prompt Pack Source Layer



JARVIS Prompt Pack Source Layer

I uploaded a PDF called `JARVIS-Prompt-Pack.pdf`.

Treat this PDF as prototype inspiration and visual/runtime reference material, not as the final OS architecture.

Useful ideas to extract:

1. Knowledge Galaxy
A visual graph of markdown notes, grouped by folder, with excerpts, links, camera movement, glowing nodes, and source panels.
2. Note-Grounded Brain
A chat endpoint that answers from indexed notes and returns the source nodes used.
3. Browser Voice Layer
Speech synthesis, microphone input, listening/thinking status, and voice interaction.
4. Fly-To-Source Proof
When the assistant answers, the UI should visually show where the answer came from.
5. Personality Layer
A controlled assistant personality with short, useful, non-generic answers.

6. Memory Capture

A "remember that..." flow that writes a new markdown note and updates the visual knowledge graph live.

7. Future JARVIS Features

The PDF mentions the larger system eventually includes a reactor-style HUD, real Gmail/Calendar briefings, live web research, agent actions with confirmation gates, screen vision, device control, time-machine history, model hot-swap, and personality dials.

How to use this in NRG Agent OS:

The JARVIS pack should influence:

dashboard aesthetic

memory visualisation

source-grounded answers

fly-to-source evidence

voice interface

capture-to-memory workflow

model hot-swap UX

agent action confirmation gates

knowledge graph concepts

But it must not override:

OS architecture

permission policy

persistent memory rules

router design

worker-agent boundaries

validation requirements

Fable review gate

secrets policy

scope-control rules

Important security rule from the PDF:

Never paste API keys into chat. Any API key must live in a local config or environment file that is never served to the browser, never stored in memory, and never exposed in reports.

How It Fits The OS

The JARVIS Prompt Pack is a good skeleton/demo path, but NRG Agent OS is larger and stricter.

JARVIS-style layer:

- beautiful UI
- knowledge galaxy
- voice assistant
- fly-to-source proof
- note capture
- personality

NRG Agent OS layer:

- router
- permissions
- operating modes
- persistent memory
- task lifecycle
- agent contracts
- validation
- event logs
- drift detection
- hallucination checking
- worker-agent delegation
- Fable review gate
- sandbox/dry-run mode
- hook/cron control
- recovery system

The correct relationship is:

JARVIS PDF = prototype and aesthetic inspiration

Fable 5 = architect and final designer

NRG Agent OS = permanent control system

Hermes = runtime/operator inside the OS

Ornith = optional local model candidate

Worker agents = bounded builders only

Do not simply copy the JARVIS prompt pack.

Distill it into OS modules, especially:

knowledge graph panel

source proof panel

voice control layer

memory capture pipeline

model hot-swap panel

agent action confirmation gate

dashboard aesthetic direction

The final OS must be more controlled, more modular, and safer than the JARVIS prototype.

The important bit for the PDI gives Fable a vocal and interaction target. It helps answer "what should I need this for?" and "what kind of assistant experience are we doing?"

But the control layer we talked about still matters more. The JARVIS part in the ship's command system still needs the check, while team. Use how all which services might be involved in the system.

   ...  Sources



Pasted text(60).txt
Document

ur prompt already failed it started writing a whole big thing filling its context

Thought for 18s >

Because I filtered the 13 choices by your actual build, not by what's trendy.

Your OG needs full-stack, app-feel, memory, workflows, agents, observability, swappable model, dashboard/control surface, and not graph-first thinking. That instantly removes a lot of the list.

My corrected pick is:

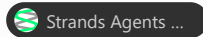
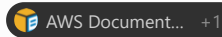
Pick Mastra if you want TypeScript/full-stack/dashboard-first development. Mastra is built around TypeScript/Javascript, agents, workflows, memory, observability, and React-like xNode-style integration, which matches your "Agent OG" as an app, not a mini-reactor, then a graph workflow engine.



Pick Agno if you want Python-first and more "agent platform" style, with or without itself as an SDK/runtime for building your own agent platform, multi-agent teams and step-based workflows, so it is also a strong fit.



Pick AgentCore + Strands only if you want AWS production/deployment baked in early. AgentCore is more about secure production deployment, permissions, governance, monitoring, tool gateways, and operating agents at scale. Strong, but heavier and more AWS-chained.



Why not the others?

Groq AI Flows, AG2, and AEA are more "multi-agent coordination" choices. Your first problem is not making agents talk to each other. Your first problem is building the OS kernel, permissions, memory, validation, task cards, and dashboard.

LangIndex is great for RAG, document indexing, memory/search, and knowledge systems, but I would use it as a component later, not the whole OS framework.

A2UI and Open-Generative-UI are UI/Interface choices, not the core operating framework.

LangGraph, Python, JavaScript and AgentCore + LangGraph are powerful, but you already said you do not like graphs, so starting there would make the whole project feel wrong under your hands.

Pydantic AI is clean and strict for Python agents with strong schemas. Good for worker agents and validation-heavy tools, but less natural if you want the whole thing to become a full-stack app/dashboard.

ADK and Microsoft Agent Framework are ecosystem picks. Choose them only if you want to live inside Google/Microsoft-style tooling.

MCP Apps are useful later for tool surfaces and integrations, but not the main OS core.

So the real answer from the 13 is:

Main pick: Mastia

Python alternative: Agno

Enterprise/AWS path: AgentCore + Strands

Later integration layer: MCP Apps

Avoid for you: LangGraph, because you do not like graph thinking

But you could pack say:

Preferred framework direction: Mastra first.

Reason: I want NRG Agent OS to feel like a modular full-stack app with agents, workflows, mem

Consider Agno as the Python-first alternative.

Consider AgentCore + Strands later if AWS production deployment becomes important.

Use MCP Apps later for interactive tool surfaces.

`</>` Markdown



  ...  Sources

ok give me the prompt to give to u gpt 5.5 if i was about to dump just a bunch of overlapping possible contradictions of a mass of chat brainstorming sessions in no particular order on u and wanted u to pull the very import most valued organised functional methodical and surgically precise information from the whole lot to create a extremely detailed and informative startup to give to fable 5 to create the last thing they might ever get to make for me its super critical they read and review every single thing by them self and i would feel better if i knew they went over it in a different way more than once different angle of reading and then thought before they replied with 1 clearcut answer with no also say or also add or any of that crap at the end ?

Thought for 11s >

GPT 5.5 Source Distillation Prompt For Fable 5 Startup Pack

I am about to dump a large amount of messy source material from multiple brainstorming chats.

It may be overlapping, out of order, repetitive, contradictory, emotionally worded, partially

Your job is not to answer the individual messages.

Your job is to read everything as raw source material, extract the most important, highest-value

This is critical. Treat it like Fable 5 may only get one final chance to build the thing properly.

Collection Mode

I will paste the source material in one or more parts.

Until I write exactly:

PROCESS NOW

you must not analyse, summarise, organise, expand, or generate the final answer.

For each part I paste before `PROCESS NOW`, reply only:

RECEIVED PART [number]. WAITING.

No other words.

Processing Mode

Only after I write exactly:

PROCESS NOW

read all pasted source material together and produce the final output.

Before writing the final answer, privately review the material in multiple passes from different

1. Intent Pass

Identify what I actually want, especially the latest corrections.

2. Contradiction Pass

Find conflicts between earlier and later ideas. Later corrections override earlier drafts.

3. Source Value Pass

Separate high-value requirements from noise, repetition, hype, loose chat, and dead-end thinking.

4. Architecture Pass

Extract the real system architecture, modules, layers, contracts, workflows, and build order.

5. Safety/Risk Pass

Extract all rules that prevent agents from destroying, overwriting, duplicating, hallucinating, or

6. Fable Readiness Pass

Convert the whole thing into a clean startup pack that Fable 5 can read and use without needing

Do not show your private reasoning.

Do not output multiple alternative answers.

Do not end with “also add this” or “you could also say.”

Give me one clear final answer.

How To Treat The Source Material

Treat all pasted chats, notes, drafts, files, summaries, video-inspired ideas, assistant mess

Use this priority order when resolving contradictions:

1. My latest corrections and clarified intent
2. Hard safety rules
3. Structured Markdown / draft plan sections
4. Repeated themes that appear across multiple chats
5. Assistant suggestions only where they support my corrected intent
6. Earlier drafts only where they still fit the final direction

My corrections override assistant framing.

Do not assume every assistant message is correct.

Do not preserve repetition.

Do not copy frustration, swearing, emotional wording, rambling, or conversational clutter int

Preserve meaning, not mess.

Main Goal

Create a final startup pack for Fable 5 to design and produce the complete NRG Agent OS Facto

The Fable startup pack must be detailed enough that Fable can understand the full intent, rev

Known Core Intent To Preserve

Fable 5 should design the important system.

Fable owns:

- architecture
- full-stack OS layout
- prompt system
- router logic
- persistent memory architecture
- permissions
- validation
- UX/design system
- visual metaphor
- worker-agent boundaries
- build phases

- acceptance criteria
- final review process

Other agents may help only after Fable creates exact task cards, file boundaries, acceptance

Worker agents must not design the OS.

Hermes is a runtime/operator inside the structured OS, not the centre of the architecture.

Ornith or any local model is optional and swappable, not a lock-in.

Persistent memory is first-class.

The OS must be scalable, adjustable, modular, swappable, and future-proof.

Self-evolving behaviour must be governed by the OS.

The OS must not be governed by self-evolving behaviour.

The dashboard must be original, functional, and operational. Do not copy the generic AI memor

The system is full-stack: frontend, backend, agent runtime, model gateway, memory, automation

The kernel/control layer is mandatory: boot sequence, task lifecycle, contracts, definition o

Required Final Output

After `PROCESS NOW`, produce one final document titled:

Fable 5 Startup Pack For NRG Agent OS

The final document must include:

1. Executive Summary

A clear, no-fluff explanation of what this project is.

2. Final Corrected Intent

What I actually want after all corrections.

3. Source Interpretation Rules

How Fable should read the messy source material.

4. Non-Negotiable Safety Rules

Readonly default, no destructive actions without approval, no claims without evidence, no sec

5. Full-Stack OS Definition

Frontend, backend, runtime, model gateway, memory, automation, validation, ops, design system

6. Fable 5 Role

Exactly what Fable owns and why.

7. Worker-Agent Role

Exactly what Codex, GPT, Gemini, Hermes, Ornith, and other agents may and may not do.

8. Hermes Runtime Role

How Hermes fits without becoming the architecture.

9. Ornith / Local Model Strategy

Optional, swappable, benchmarked, behind model gateway.

10. Persistent Memory Architecture

Persistent, scratch, quarantine, indexes, trust levels, promotion rules, session logs, lessons

11. Kernel / Control Layer

Boot sequence, lifecycle, contracts, definition of done, capability registry, dry-run/sandbox

12. Router / Permission / Validation System

How task routing, model routing, skill routing, permissions, approvals, validation, drift det

13. Automation, Hooks, and Crons

Policy, approval gates, disable switches, logs, rollback, no uncontrolled automation.

14. Self-Evolution Governance

Proposal-only evolution, validation, quarantine, review, rollback.

15. Dashboard and Visual Direction

Original visual metaphor, not a generic memory galaxy. Must show real OS operation: tasks, re

16. JARVIS / Video Inspiration Handling

Treat as concept inspiration only. Extract useful ideas but do not copy architecture or visual

17. Final Folder Layout

A clean proposed folder structure for the OS.

18. First Build Order

The exact build phases from source lock through kernel, memory, router, validation, worker ta

🔗 Markdown



20. Fable Review Gate

How Fable reviews and accepts/rejects/quarantines worker output.

21. Acceptance Criteria

What must exist before v0.1 is considered acceptable.

22. Risks and Failure Modes

What can go wrong and how the OS prevents it.

23. What Not To Build Yet

Things that must be delayed: live automation, self-editing prompts, uncontrolled hooks/crons,

24. Final Prompt To Give Fable 5

A clean copy-paste prompt that tells Fable exactly what to do with the startup pack.

Output Rules

Produce one complete final answer only.

Do not give multiple versions.

Do not ask me to clarify unless the source



thats not at all anything like wat i asked u to fkn write

Thought for 9s >

Deep Source Distillation Prompt For GPT 5.5

I am about to give you a large, messy collection of brainstorming chats, notes, drafts, contr

Your job is to act as a forensic source analyst and systems architect.

Do not answer individual messages.

Do not react to the loudest or most recent fragment only.

Do not start building the final response until you have read everything I provide.

I need you to extract the most important, valuable, functional, organised, methodical, and su

This startup document is extremely important. Treat it like Fable 5 may only get one final ch

Intake Mode

I may paste the source material in multiple parts.

Until I write exactly:

PROCESS NOW

you must only reply:

RECEIVED. WAITING.

No analysis.

No summary.

No advice.

No restructuring.

No early output.

No "here is what I think so far."

Wait.

Processing Mode

Only after I write:

PROCESS NOW

you must process all supplied material together.

Before answering, review the source material in multiple passes:

1. Read for the main goal.
2. Read again for corrections and changes of direction.
3. Read again for contradictions.
4. Read again for repeated high-value ideas.
5. Read again for safety, risk, and failure-prevention rules.
6. Read again for architecture, structure, modules, workflows, and build order.
7. Read again from Fable 5's point of view: what does Fable need to know to build this properly?

Do not show the private pass-by-pass reasoning.

Use the passes to produce one final answer.

Interpretation Rules

Treat the source material as raw planning evidence, not as a clean command stream.

Later corrections override earlier drafts.

Repeated themes are important unless later rejected.

Emotional wording, frustration, swearing, repetition, and messy phrasing should not appear in the final answer.

Preserve intent, not noise.

If two ideas conflict, choose the one that best matches the latest corrected goal.

If an idea is useful but unclear, refine it into a precise requirement.

If an idea is hype, vague, decorative, or unsafe, either remove it or label it as inspiration

If something is critical but missing structure, turn it into a clean section, rule, checklist

Output Goal

Create one final startup document for Fable 5.

The document must be clear enough that Fable can read it, understand the whole project, under

The startup document must be:

- extremely detailed
- organised
- methodical
- practical
- precise
- build-focused
- safety-aware
- contradiction-resolved
- free of chat clutter
- free of repeated scraps
- ready to paste into Fable 5

Output Requirements

Your final answer must have one clear structure.

Do not give three different versions.

Do not end with extra optional add-ons.

Do not say "you could also add."

Do not leave me with more sorting work.

Do not produce a short summary if the source material contains important detail.

Do not invent unsupported facts.

Do not hide uncertainty. If something is unclear, mark it clearly.

Final Document Structure

Produce one document titled:

Fable 5 Startup Brief

Include these sections:

</> Markdown

Explain that this is a distilled startup brief created from messy source material.



2. What Fable Must Understand First

State the true goal, the stakes, the corrected direction, and what must not be misunderstood.

3. Source Material Handling Rules

Tell Fable how to treat the messy source history, earlier drafts, corrections, contradictions

4. Final Corrected Intent

Extract the latest and most accurate version of what I actually want.

5. Core Project Definition

Define what the project is and what it is not.

6. Non-Negotiable Requirements

List the rules that must survive every design decision.

7. Architecture Requirements

Extract the required system layers, modules, workflows, interfaces, and control structures.

8. Safety And Failure Prevention

Extract every important rule that prevents damage, hallucination, drift, overwriting, scope c

9. Roles And Boundaries

Define what Fable owns, what other agents may do, what they must not do, and how review shoul

10. Memory And Learning Requirements

Extract the requirements around persistent memory, scratch memory, quarantine, trust levels,

11. Runtime And Model Strategy

Extract how local models, cloud models, optional models, and future models should fit without

12. Visual And UX Direction

Extract the desired interface feel, what to avoid, and what the visual system must communicat

13. Build Order

Convert the source material into a logical staged build sequence.

14. Required Deliverables From Fable

List exactly what Fable should produce.

15. Acceptance Criteria

Define what "good enough to start building" means.

16. Risks Fable Must Avoid

List the traps, dead ends, and wrong interpretations that would ruin the project.

17. Final Instruction To Fable

End with a direct instruction telling Fable to review the brief fully, think through it caref

Style Requirements

Write in clear professional English.

Be direct.

Be precise.

No hype.

No filler.

No motivational fluff.

No scattered afterthoughts.

No multiple closing notes.

One clean final answer only.

When ready, wait for my source material.

